MEAL-62

Complete MVP API Contract & Implementation Guide

Meal Connect | Angular 21 + TypeScript | Python FastAPI | PostgreSQL | Localhost, without Docker

Version 1.3 • API contract for implementation • Related: MEAL-5 v1.3, MEAL-61 v1.3, MEAL-6 database model

1. Purpose and implementation boundary

This is the canonical API contract for the Meal Connect MVP. The Angular team implements the request/response types written here; the FastAPI team implements these HTTP routes and validates permissions, state transitions and database invariants. This document defines the target contract; it does not claim the endpoints are already implemented.

Core flow: approved restaurant/bakery publishes a donation → verified NGO requests a quantity from that donation → owning business approves the entire request or rejects → volunteer pickup → NGO distribution record. No general NGO request board, partial business approvals, payment, live GPS or Docker.

2. Naming, types and transport

Convention	Rule
API base	/api/v1; localhost example http://localhost:8000/api/v1
Content type	application/json for request/response; UTF-8
Paths	lowercase plural resource names; IDs in path, e.g. /donations/{donation_id}; workflow actions /requests/{request_id}/approve
JSON names	snake_case; IDs are integer; quantities are decimal numbers with at most 2 fraction digits; quantity unit is set by donation
Dates	timezone-aware ISO 8601 strings; server stores PostgreSQL timestamptz; API emits UTC offsets (+00:00 / Z)
Pagination	page=1, page_size=20 by default, maximum page_size=100; total_items and total_pages returned
Sorting	sort=pickup_deadline or sort=-created_at; only documented fields; stable secondary sort by primary key
Authentication	Authorization: Bearer <access_token>; user identity and scoped role validated on FastAPI
CORS	allow exact Angular localhost origin, typically http://localhost:4200; no wildcard credentials
Canonical success	JSON object {"data": ...} (plus "pagination" for lists); no success message required
Canonical errors	JSON {"error":{"code":...,"message":...,"details":[],"request_id":...}}; FastAPI exception handlers required

3. HTTP methods and status codes

HTTP	Contract

GET / 200	Read one or list; no mutation
POST / 201	Create resource; return created representation
POST action / 200	Business workflow transition (approve, reject, cancel, deliver, etc.)
PATCH / 200	Update permitted fields, return current resource
401	Missing/invalid/expired bearer authentication
403	Authenticated but role, organization membership, verification or assignment not permitted
404	Entity absent or intentionally undisclosed
409	State transition/expiry/stock race/duplicate operation conflict
422	Invalid JSON fields, IDs, filters or Pydantic validation
500	Unexpected error; safe response and server-side trace
503	Optional: database/service temporarily unavailable

4. Error envelope and validation mapping

{"error":{"code":"VALIDATION_ERROR","message":"Please correct the highlighted fields","details":
[{"field":"requested_quantity","message":"Must be greater than zero"}],"request_id":"req_8f12"}}

Required: error.code (UPPER_SNAKE_CASE), error.message (safe text), error.details (array), error.request_id (non-secret correlation string). For non-field errors details=[]. FastAPI must transform default Pydantic/FastAPI validation exceptions to this same envelope. Never expose stack traces, password hashes, access tokens or SQL errors.

Error code	HTTP	Client response
VALIDATION_ERROR	422	Attach details to matching Angular reactive form controls
AUTH_REQUIRED	401	Clear auth state / present login
FORBIDDEN	403	Show access restriction
NOT_FOUND	404	Show not found
QUANTITY_CONFLICT	409	Refresh donation availability
DONATION_EXPIRED	409	Refresh donation state
INVALID_STATE_TRANSITION	409	Refresh request or pickup
ALREADY_EXISTS	409	Show duplicate warning
INTERNAL_ERROR	500	Generic error; show request_id

5. Standard list query and response

GET /api/v1/donations?
city=Colombo&category=prepared_meal&min_quantity=5&sort=pickup_deadline&page=1&page_size=20

{"data":[{"donation_id":15,"title":"Rice
packets","total_quantity":50,"allocated_quantity":10,"quantity_unit":"portions","status":"partially_allocated"}],"pagination":
{"page":1,"page_size":20,"total_items":1,"total_pages":1}}

Supported donation filters: city, category, status, min_quantity (minimum remaining availability), available_before (ISO 8601); supported sorts: pickup_deadline, -pickup_deadline, created_at, -created_at. Exclude expired/cancelled

donations from public discovery by default. Invalid page/page_size, filter values or sort keys → 422. Scoped lists must never reveal other organizations' private actions.

6. Authentication and organizations

MVP baseline: bearer JWT access token, one signed-in global role per user (admin, business_staff, ngo_staff, volunteer). A user's organization access further requires an ACTIVE organization_memberships row with the corresponding organization type and permitted membership_role. Do not store JWTs in logs. Token expiry and refresh handling are security configuration, not a dependency on a specific UI component. For localhost MVP a short-lived access token and re-login on expiry is the defined minimum; no refresh-token endpoint required.

6.1 Authentication and organization routes

Method / route	Role	Body / query	Success / behavior
POST /auth/register	Public	{first_name,last_name,email,password,phone?,role}	201 {user_id,first_name,last_name,email,role,account_status}; Business/NGO membership is not automatic; role must be one of business_staff, ngo_staff, volunteer; no public admin signup.
POST /auth/login	Public	{email,password}	200 {access_token,token_type:"bearer",expires_in,user:{user_id,role,email}}; Bad credentials → 401.
GET /users/me	Signed-in	—	200 user profile with user_id,first_name,last_name,email,role,account_status; Never return password_hash.
POST /organizations	Business/NGO staff	{name,type,registration_number?,contact_email,contact_phone?,address_line,city,district?}	201 organization record; Creates pending org and active owner membership for actor atomically; org type must match global role.
GET /organizations/me	Business/NGO staff	—	200 {data:[organization summaries]}; Returns organizations in which caller is active member.
POST /organizations/{organization_id}/verification-submission	Org owner/manager	{}	200 organization summary; Submit pending verification; cannot self-approve.
GET /admin/organizations?verification_status=pending	Admin	page,page_size	200 paginated organizations; Filter by verification_status.
POST /admin/organizations/{o	Admin	{decision:"approved"|"rejected"|"suspended",revie	200 organization summary; Persist

rganization_id}/verification-decision		w_note?}	organization current state + organization_verifications row + status_history atomically.

7. Donations and NGO requests

Method / route	Authorized role	Request or query	Response
GET /donations	Signed-in user	Filters: city,category,status,min_quantity,available_before,page,page_size,sort	200 paginated donation summaries
GET /donations/{donation_id}	Signed-in user	—	200 donation details, including remaining_quantity computed by server
POST /donations	Approved restaurant/bakery member	{organization_id,title,category,description?,total_quantity,quantity_unit,allergen_info?,storage_instructions?,prepared_at?,available_from,pickup_deadline,pickup_address,pickup_city,status:"draft"|"available"}	201 donation; allocated_quantity=0 is server-set
PATCH /donations/{donation_id}	Owning approved business	Permitted: title,category,description,allergen_info,storage_instructions,pickup_address,pickup_city; quantity/deadline changes only if no approved allocations	200 updated donation; validate lifecycle
POST /donations/{donation_id}/publish	Owning approved business	{}	200 donation with status=available; validate all required fields
POST /donations/{donation_id}/cancel	Owning business	{reason?}	200 donation status=cancelled; reject if pickup/approved allocation prevents safe cancellation
POST /donations/{donation_id}/requests	Approved NGO member	{ngo_organization_id,requested_quantity,request_note?}	201 request with status=pending; validate same unit, deadline, non-expired listing
GET /requests/me	Approved NGO member	{status?,page?,page_size?}	200 paginated NGO requests limited to active NGO memberships
GET /donations/{donation_id}/requests	Owning business	{status?,page?,page_size?}	200 paginated requests for business-owned donation

POST /requests/{request_id}/approve	Owning business	{decision_note?}	200 approved request with exact original requested_quantity
POST /requests/{request_id}/reject	Owning business	{decision_note?}	200 rejected request; no quantity reservation
POST /requests/{request_id}/cancel	Requesting NGO	{reason?}	200 cancelled request; if approved and before collection, release allocation once transactionally

7.1 Critical request and approval payloads

POST /donations/15/requests → 201 Created

Request: {"ngo_organization_id":8,"requested_quantity":20,"request_note":"For evening distribution"}
Response: {"data":
{"request_id":42,"donation_id":15,"ngo_organization_id":8,"requested_quantity":20,"status":"pending","created_at":"2026-11-01T09:00:00Z"}}

POST /requests/42/approve → 200 OK

Request: {"decision_note":"Collection approved"}
Response: {"data":{"request_id":42,"donation_id":15,"requested_quantity":20,"status":"approved","reviewed_at":"2026-11-01T09:05:00Z"}}

No approved_quantity is accepted. FastAPI locks the pending request and target donation; checks business ownership, NGO verification, listing status, deadline and remaining quantity; increments allocated_quantity and changes request status in ONE PostgreSQL transaction; writes status_history, then commits. Any failed check rolls back; 409 QUANTITY_CONFLICT or INVALID_STATE_TRANSITION. Keep donation status available/partially_allocated/fully_allocated synchronized with allocation.

8. Pickups and distributions

Method / route	Authorized role	Body / query	Success
POST /pickups	Business owner or Admin	{request_id,collector_user_id?}	201 pickup, status=assigned; one per approved request
GET /pickups/me	Volunteer	{status?,page?,page_size?}	200 assigned pickups only
GET /pickups/{pickup_id}	Assigned volunteer, related business/NGO, admin	—	200 pickup with permitted linked request summary
PATCH /pickups/{pickup_id}	Assigned volunteer	{scheduled_at?,pickup_notes?}	200 pickup; status scheduled when schedule set; deadline validation
POST /pickups/{pickup_id}/picked-up	Assigned volunteer	{picked_up_at?,pickup_notes?}	200 status=picked_up; server timestamp default
POST /pickups/{pickup_id}/delivered	Assigned volunteer	{delivered_at?,delivery_notes?}	200 status=delivered; previously picked_up required
POST /pickups/{pickup_id}/fail	Assigned volunteer	{reason}	200 status=failed; audit status

ed			
POST /distributions	Related verified NGO member	{pickup_id,distributed_quantity,beneficiary_count,distributed_at,notes?}	201 final distribution; one per delivered pickup
GET /distributions/me	NGO member	{page?,page_size?}	200 paginated distribution records for member NGO

A request has zero or one pickup (UNIQUE pickups.request_id), and a pickup has zero or one final distribution (UNIQUE distribution_records.pickup_id). Verify distributed_quantity > 0 and <= approved request quantity, beneficiary_count >= 0, and submitted quantity unit implicitly equals the source donation unit. After distribution, set related request status=fulfilled atomically with a status_history entry. Never permit distribution before delivered pickup.

9. Notifications, audit and reports

Method / route	Access / body	Success
GET /notifications/me	Signed-in; page,page_size,is_read?	200 paginated recipient-only notifications
PATCH /notifications/{notification_id}	Recipient; {is_read:true}	200 notification; read_at set by server
GET /reports/summary	Admin; from?,to? ISO dates	200 {donations_created,requests_approved,pickups_delivered,distributed_quantity_by_unit,beneficiary_count}
GET /status-history/{entity_type}/{entity_id}	Admin or actor authorized for entity	200 paginated status changes; entity_type restricted to organization,donation,request,pickup

Do not sum quantities across unlike units (e.g. kg plus portions). Reports derive from durable database records, not frontend counters. Status-history and notification polymorphic entity_type/entity_id references require backend target validation.

10. End-to-end examples and invariant tests

Test scenario	Required API behavior
Business is not verified	POST /donations publish → 403 FORBIDDEN
NGO tries general request	No general /food-requests POST endpoint; only /donations/{id}/requests
Zero or negative requested quantity	422 VALIDATION_ERROR with details.field=requested_quantity
Donation expired at request time	409 DONATION_EXPIRED
Two competing approvals exceed stock	At most one approval commits; other 409 QUANTITY_CONFLICT
Same pending request approved twice	Second 409 INVALID_STATE_TRANSITION
Approval request includes approved_quantity	422 extra field forbidden; never partial approval
Volunteer edits another person's pickup	403 FORBIDDEN
Distribution before delivery	409 INVALID_STATE_TRANSITION

Second pickup for same request	409 ALREADY_EXISTS; database UNIQUE enforced
Cross-organization data access	403 or 404; no data leakage

11. Angular 21 and FastAPI implementation rules

- Angular: centralize typed requests in injectable HttpClient services; expose API base URL via environment config. Do not send password_hash, server-managed created_at, allocated_quantity or reviewer IDs from clients.
- Angular: HTTP interceptor attaches bearer token only to trusted API origin; standard error parser maps 422 details to reactive forms, and 409 to refresh-and-retry UI. Route guards are UX only, not authorization.
- FastAPI: route controllers use Pydantic request/response schemas with extra="forbid" for mutation inputs, service layer for business logic, repositories/transaction boundaries for PostgreSQL, and centralized exception handlers.
- FastAPI: authenticated user and organization membership resolved server-side; never trust organization_id or user_id in request alone. Return no records outside scope.
- FastAPI: status_history inserts occur with their related entity transition. Server computes remaining_quantity = total_quantity - allocated_quantity; do not store an independently mutable remaining_quantity column.
- Localhost: Angular normally http://localhost:4200; FastAPI normally http://localhost:8000; FastAPI /docs exposes implemented OpenAPI; PostgreSQL runs locally. No Docker requirement.
- Keep enum values identical to MEAL-6 DBML: donation draft/available/partially_allocated/fully_allocated/collected/expired/cancelled; request pending/approved/rejected/cancelled/fulfilled; pickup assigned/scheduled/picked_up/delivered/failed/cancelled.
- Changes to route names, payload fields or response schemas must be applied together to MEAL-62, FastAPI OpenAPI and Angular TypeScript types; no undocumented endpoint drift.

12. Source-of-truth and build sequence

MEAL-5 v1.3 defines system boundaries; MEAL-61 v1.3 defines component flow; MEAL-6 defines PostgreSQL entities, identifiers, constraints and status enums; THIS document defines the target HTTP API. MEAL-63 and MEAL-163 define environment versions and configuration. Implement in sequence: schemas/exception handlers → authentication/organization membership → donations/requests → transactional approvals → pickup/distribution → notifications/reports → Angular integration tests.

Clarifications for implementation: The routes, JWT access-token baseline, and pagination defaults above are the selected v1.3 project contract, not claims of deployed functionality. Any future change should be reflected in this document and the code together. Avoid adding out-of-scope features without updating MVP scope.